

# UT2 - Práctica 1: IDE y Depuración

**Alumno:** Juan Manuel Barbosa Camacho | **Fecha:** 21/01/2026 |

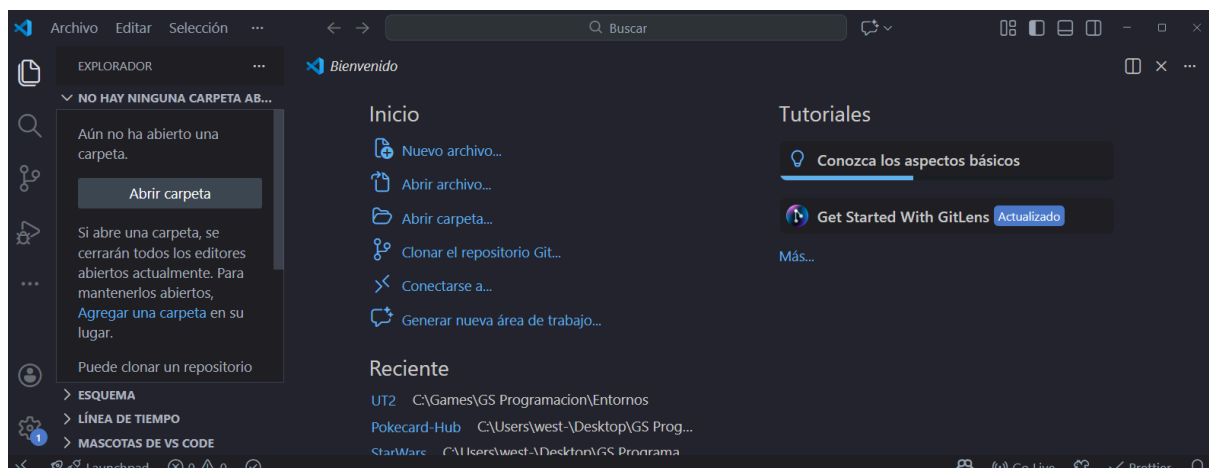
**Módulo:** Entornos de Desarrollo - 1º DAW

## Actividad 1: Instalación de Visual Studio Code

He descargado VS Code desde <https://code.visualstudio.com/>. La web detecta automáticamente el sistema operativo.

### Proceso en Windows:

1. Descargar el instalador .exe
2. Ejecutar y aceptar términos
3. Marcar opciones: icono en escritorio y "Abrir con Code" en menú contextual
4. Completar instalación



## Actividad 2: Ejecución del script\_1

He descargado "DepurarParte1.zip" y creado la carpeta "UT2\_Practica1" para trabajar.

**¿En qué lenguaje está escrito?** Python (.py, sintaxis con print, sin llaves)

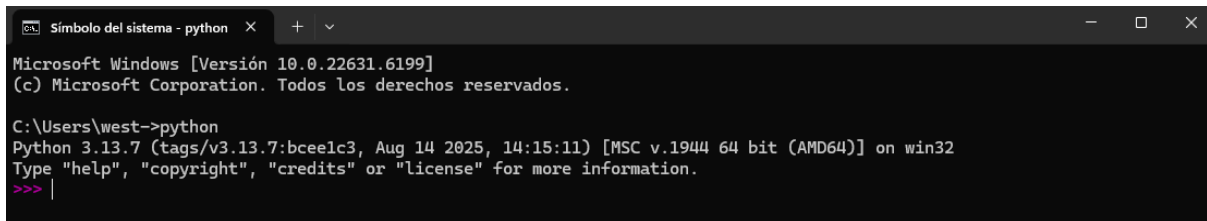
**¿Es compilado o interpretado?** Interpretado - se ejecuta línea por línea sin compilación previa.

**¿Pudiste ejecutarlo directamente?** No, necesité instalar Python primero.

## ¿Qué instalaste?

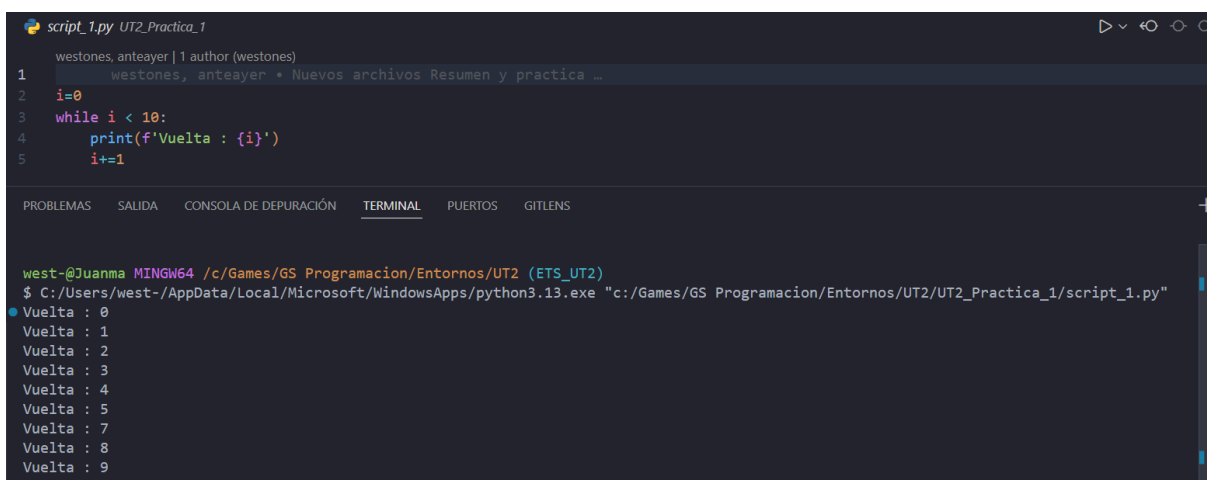
- **Python 3.12** desde python.org (marcando "Add Python to PATH")
- **Extensión Python** de Microsoft en VS Code (ms-python.python)

Verificación: **python** en terminal



```
Símbolo del sistema - python
Microsoft Windows [Versión 10.0.22631.6199]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\west->python
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```



```
script_1.py UT2_Practica_1
westones, anteayer | 1 author (westones)
1 westones, anteayer • Nuevos archivos Resumen y practica ...
2 i=0
3 while i < 10:
4     print(f'Vuelta : {i}')
5     i+=1

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  GITLENS

west@Juanma MINGW64 /c/Games/GS Programacion/Entornos/UT2 (ETS_UT2)
$ C:/Users/west-/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Games/GS Programacion/Entornos/UT2/UT2_Practica_1/script_1.py"
Vuelta : 0
Vuelta : 1
Vuelta : 2
Vuelta : 3
Vuelta : 4
Vuelta : 5
Vuelta : 6
Vuelta : 7
Vuelta : 8
Vuelta : 9
```

## Actividad 3: Características de Python y Plugins

### Puntos Fuertes:

- Sintaxis clara y simple
- Gran ecosistema de librerías (NumPy, Pandas, Django, TensorFlow)
- Multiplataforma y versátil

### Puntos Débiles:

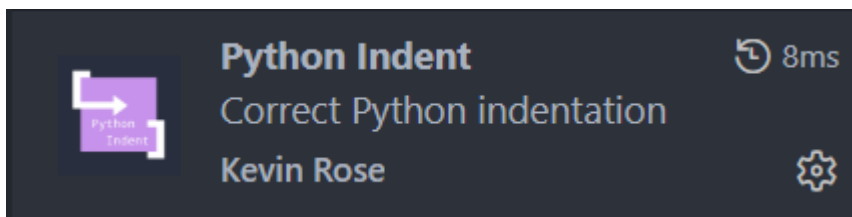
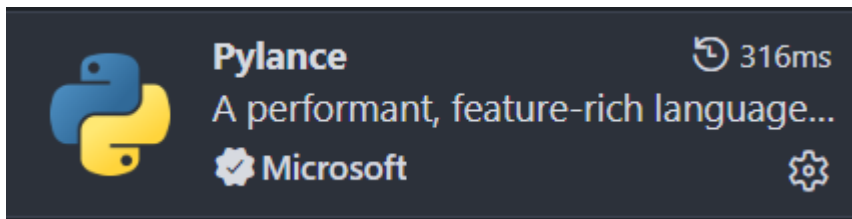
- Más lento que lenguajes compilados
- Mayor consumo de memoria
- Errores de tipo solo en tiempo de ejecución

### Campos de Aplicación:

- Ciencia de datos e IA/ML
- Desarrollo web (Django, Flask)
- Automatización y scripting
- DevOps

### Plugins instalados:

1. **Pylance** - Mejora IntelliSense con mejor autocompletado y detección de tipos
2. **Python Indent** - Corrige automáticamente la indentación (crucial en Python)



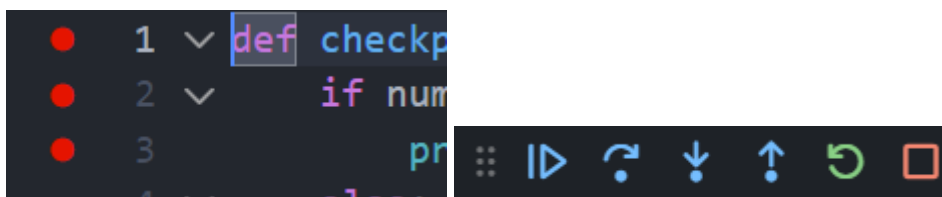
## Actividad 4: Depuración y Breakpoints

### Conceptos básicos:

- **Breakpoint:** Punto rojo donde se detiene la ejecución (clic izquierda del número de línea)
- **Modo Debug:** Permite ejecutar código paso a paso y ver valores de variables

### Opciones de ejecución:

- **Continue (F5):** Ejecuta hasta el siguiente breakpoint
- **Step Over (F10):** Ejecuta línea actual sin entrar en funciones
- **Step Into (F11):** Ejecuta y entra dentro de funciones
- **Step Out (Shift+F11):** Sale de la función actual

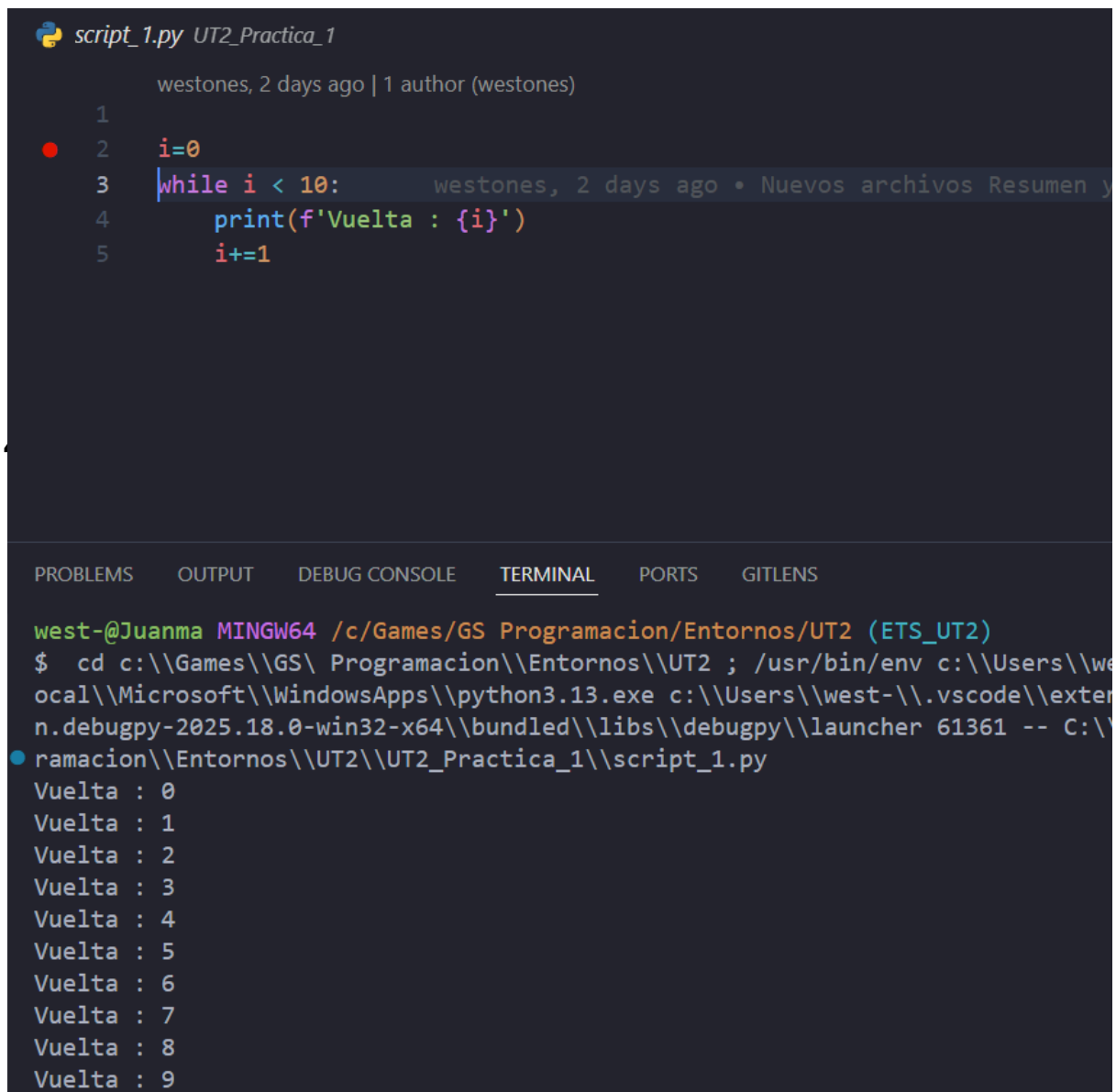


#### 4.1. Script\_1.py

```
1
2     i=0
3     while i < 10:
4         print(f'Vuelta : {i}')
5         i+=1
```

Breakpoint en línea 2. Al depurar:

- Variable `i` va cambiando: 0, 1, 2... hasta 9
- Step Over y Step Into funcionan igual (no hay funciones propias)
- Step Out actúa como Continue



The screenshot shows the Visual Studio Code editor with a Python script named `script_1.py` in the editor pane. The script contains a `while` loop that prints the current value of `i` from 0 to 9. A red breakpoint is set on line 2, where `i` is initialized to 0. The interface shows the file explorer on the left with the project name `UT2_Practica_1`. The bottom pane is the TERMINAL, which displays the command prompt output of running the script. The output shows the program's execution, printing 'Vuelta : 0' through 'Vuelta : 9'.

```
script_1.py UT2_Practica_1
westones, 2 days ago | 1 author (westones)
1
2     i=0
3     while i < 10:
4         print(f'Vuelta : {i}')
5         i+=1

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
west-@Juanma MINGW64 /c/Games/GS Programacion/Entornos/UT2 (ETS_UT2)
$ cd c:\\Games\\GS\\ Programacion\\Entornos\\UT2 ; /usr/bin/env c:\\Users\\we
ocal\\Microsoft\\WindowsApps\\python3.13.exe c:\\Users\\west-\\.vscode\\exter
n.debugpy-2025.18.0-win32-x64\\bundled\\libs\\debugpy\\launcher 61361 -- C:\\
ramacion\\Entornos\\UT2\\UT2_Practica_1\\script_1.py
Vuelta : 0
Vuelta : 1
Vuelta : 2
Vuelta : 3
Vuelta : 4
Vuelta : 5
Vuelta : 6
Vuelta : 7
Vuelta : 8
Vuelta : 9
```

```
script_2.py UT2_Practica_1
westones, 2 days ago | 1 author (westones)
1
2  ✓ def sacarPorPantalla(numero):
3      print(f'Vuelta : {i}')
4
5
6  ● i=0
7  ✓ while i < 10:
8      sacarPorPantalla(i)
9      i+=1 westones, 2 days ago • Nuevos a
```

Breakpoint en línea 6 (llamada a función).

#### Diferencias entre modos:

- **Step Over:** Ejecuta `sacarPorPantalla(i)` completa, no entra
- **Step Into:** Entra en la función, muestra parámetro `numero=0`
- **Step Out:** Sale de la función si estás dentro

```
5
6  i=0 westones, 2 days a
7  while i < 10:
8      sacarPorPantalla(i)
9      i+=1
```

```
1
2  def sacarPorPantalla(numero): numero = 0
3  print(f'Vuelta : {i}') i = 0 westo
4
5
6  ● i=0
7  while i < 10:
8      sacarPorPantalla(i)
9      i+=1
```

```

2  def sacarPorPantalla(numero): numero = 1
3  print(f'Vuelta : {i}') i = 1
4
5
6  i=0
7  while i < 10:
8      sacarPorPantalla(i)
9      i+=1

```

```

2  def sacarPorPantalla(numero): numero = 6
3  print(f'Vuelta : {i}') i = 6
4
5
6  i=0
7  while i < 10:
8      sacarPorPantalla(i)
9      i+=1

```

### 4.3. Script\_3.py

```

westones, 2 days ago | 1 author (westones)
1  colores = ["rojo", "amarillo", "naranja","azul","morado"]
2
3  for i in range(len(colores) + 1):
4      print(colores[i])
westones, 2 days ago • Nuevos ar

```

**Error:** IndexError: list index out of range

**Causa:** `range(len(colores) + 1)` genera 0,1,2,3,4,5. La lista solo tiene índices 0-4.

Al depurar línea por línea, cuando `i=5` → `colores[5]` no existe.

**Solución:** Eliminar el `+ 1`:

`for i in range(len(colores)):`

```

west-@Juanma MINGW64 /c/Games/GS Programacion/Entornos/UT2 (ETS_UT2)
$ C:/Users/west-/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Games/GS Programacion/Entornos/UT2/UT2_Practica_1/script_3.py"
rojo
amarillo
naranja
azul
morado
Traceback (most recent call last):
  File "c:/Games/GS Programacion/Entornos/UT2/UT2_Practica_1/script_3.py", line 4, in <module>
    print(colores[i])
    ~~~~~^~~~~
IndexError: list index out of range
west-@Juanma MINGW64 /c/Games/GS Programacion/Entornos/UT2 (ETS_UT2)

```

```

westones, 2 days ago | 1 author (westones)
1 colores = ["rojo", "amarillo", "naranja", "azul", "morado"]
2
3 for i in range(len(colores) + 1): colores = ['rojo', 'amarillo', 'naranja', 'azul', 'morado']
4 print(colores[i])

```

```

You, 3 minutes ago | 2 authors (westones and one other)
1 colores = ["rojo", "amarillo", "naranja", "azul", "morado"]
2
3 for i in range(len(colores)):
4     print(colores[i])

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```

west-@Juanma MINGW64 /c/Games/GS Programacion/Entornos/UT2 (ETS_UT2)
$ C:/Users/west-/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Games/GS Programacion/Entornos/UT2/UT2_Practica_1/script_3.py"
rojo
amarillo
naranja
azul
morado

```

## 4.4. Script\_4.py

```

script_4.py UT2_Practica_1
westones, 2 days ago | 1 author (westones)
1 colores = ["rojo", "amarillo", "naranja", "azul", "morado"]
2
3 for i in range(len(colores) + 1):
4     print(colores[i])
5     resultado=colores[i]+1

```

### Errores:

1. **IndexError** (mismo que script\_3)
2. **TypeError: can only concatenate str (not "int") to str**

**Explicación:** Intenta sumar texto (`colores[i]+1`) con número (1). Python no permite sumar tipos incompatibles.

```

west-@Juanma MINGW64 /c/Games/GS Programacion/Entornos/UT2 (ETS_UT2)
$ C:/Users/west-/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Games/GS Programacion/Entornos/UT2/UT2_Practica_1/script_4.py"
rojo
Traceback (most recent call last):
  File "c:/Games/GS Programacion/Entornos/UT2/UT2_Practica_1/script_4.py", line 5, in <module>
    resultado=colores[i]+1
    ~~~~~^~~~~
TypeError: can only concatenate str (not "int") to str

```

## 4.5. Script\_5.py

```
script_5.py UT2_Practica_1
westones, 2 days ago | 1 author (westones)
1 westones, 2 days ago • Nuevos archivos Resumen y practica ...
2 ▼ def operaciones(a, b):
3     resultado_intermedio = a * 2
4     resultado_intermedio_2 = resultado_intermedio*10
5     resultado_final = resultado_intermedio + b + resultado_intermedio_2
6     return resultado_final
7
8     resultado_operacion = operaciones(5, 10)
9
10 print("Resultado final:", resultado_operacion)
```

Breakpoint en línea 7. Uso **Step Into** para entrar en la función.

Evolución de variables:

- $a=5, b=10$
- $\text{resultado\_intermedio} = 5*2 = 10$
- $\text{resultado\_intermedio\_2} = 10*10 = 100$
- $\text{resultado\_final} = 10+10+100 = 120$

```
▼ Locals
a = 5
b = 10
resultado_final = 120
resultado_intermedio = 10
resultado_intermedio_2 = 100
```

He añadido `resultado_final` a Watch para seguimiento.

```
▼ WATCH
resultado_final = 120
```



## 4.6. Script\_6.py

Similar al script\_5 pero con `input()` del usuario.

**Pruebas:**

| Entrada | Resultado                               |
|---------|-----------------------------------------|
| 10, 5   | ✓ Resultado: 225                        |
| abc     | ✗ ValueError: invalid literal for int() |
| -5, 3   | ✓ Resultado: -107                       |

### ¿Qué pasa con letras?

Al introducir `abc`, Python intenta hacer `int("abc")` y genera un **ValueError** porque no puede convertir letras a número. El programa se detiene y muestra el error.

```
west-@Juanma MINGW64 /c/Games/GS Programacion/Entornos/UT2 (ETS_UT2)
$ C:/Users/west-/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Games/GS Programacion/Entornos/
py"
Introduce el primer operando: abc
Traceback (most recent call last):
  File "c:\Games\GS Programacion\Entornos\UT2\UT2_Practica_1\script_6.py", line 9, in <module>
    p1 = int(input("Introduce el primer operando: "))
❖ValueError: invalid literal for int() with base 10: 'abc'
```

Para evitar que el programa se rompa, habría que validar la entrada antes de convertirla

## 4.7. Script\_7.py

Programa que verifica si un número es par/impar y pregunta si continuar.

**Pruebas:**

- `8 + y` → Imprime "Par!" y termina ✓
- `7 + n` → Imprime "Impar!" y continúa ✓
- `Y` (mayúscula) → Funciona correctamente ✓
- `hola` → Muestra "No has introducido un número" ✓
- Opción inválida (`x`) → Pide "Indica 'y' o 'n'" hasta entrada válida ✓

¿Funciona con mayúsculas? Sí, usa `.lower()` en todas las comparaciones (líneas 8, 16, 18, 23).

**Conclusión:** El código está **bien diseñado** - no hay errores que corregir. Incluye validación con try-except y manejo de mayúsculas.

```
Introduce un número: 8
El número es Par!
Quieres salir? y/n
y
```

```
Introduce un número: 7
El número es Impar!
Quieres salir? y/n
n
```

```
Introduce un número: hola
No has introducido un número
Quieres salir? y/n
```

```
No has introducido un número
Quieres salir? y/n
x
Indica 'y' o 'n' x
```

## Conclusiones

He aprendido a:

- Instalar y configurar VS Code para Python
- Usar breakpoints y modo debug
- Diferenciar Step Over, Step Into, Step Out y Continue
- Identificar errores comunes: IndexError, TypeError, ValueError
- Observar variables durante la ejecución

La depuración es fundamental para encontrar y corregir errores de forma eficiente.